

From answering to acting: building a B2B SaaS support agent

In my last project I built a RAG system — it *answered questions* from a documentation corpus. This time I wanted something that *acts*: an agent that handles a support ticket end to end, looks up live information with tools, decides when to chain several of them, recovers when they fail, remembers the conversation, and logs exactly what it did and what it cost.

This post is the build story. I'm going to lead with the parts that were actually *interesting* — the place where two tools still couldn't answer a question, the two separate halves of "failing gracefully," and the moment the cost data made a design decision for me — rather than walk a feature list.

*A note up front on honesty: the account and entitlements tools in this project return **mock data** (canned Python dicts standing in for a real billing/account service). That's deliberate — the interesting engineering is in the agent loop, the failure handling, and the observability, none of which change when the dict becomes a real API call. I'll flag the mock boundary where it matters.*

[Figure: system architecture — see [docs/architecture.svg](#)]

The shift: a straight line becomes a loop

My RAG project was a straight line — retrieve → generate, the same two steps every time. I decided the steps. An agent is different: it's a **loop the model drives**.

```
user message
  → model decides: answer directly, OR call a tool
    → (tool call) my code runs the tool, returns the result
      → model continues with the result
        → final answer, or another tool call ...
```

The model chooses *whether* to use a tool, *which* one, and *how many times*. My code's job is to run the tools it asks for and feed the results back until it's done. I wrote this loop by hand first (raw SDK), then rebuilt the identical behaviour with the SDK's tool runner and again with a framework — three abstraction levels of the same thing — specifically so the mechanics weren't magic to me.

Reusing a whole RAG system as a single tool

The agent needed to know how the product works. I'd already built exactly that in the previous project: a retrieval system over the docs. So instead of rebuilding it, I **wrapped it as a tool**. The persisted vector index from project 1 loads lazily and becomes `search_docs(query)`.

```
def search_docs(query: str) -> dict:
    nodes = load_project1_index().as_retriever(similarity_top_k=3).retrieve(query)
    return {"results": [
        {"source": n.node.metadata.get("file_name"),
         "score": round(float(n.score), 3),
         "text": n.node.get_content()[:500]} for n in nodes]}
```

This is a small idea with a big payoff: **an entire system can become one capability inside a bigger one**. A RAG pipeline is now just a tool the agent reaches for. Nothing was rebuilt — I pointed at an index that was already on disk.

Routing vs. chaining

With more than one tool, the agent's first job is **routing** — pick the *single* right tool. A docs question (How do I enable row-level security?) goes to `search_docs`; an account question (What plan is `acct_789` on?) goes to `get_account_status`. You see it in the logs as exactly one tool call, then an answer. The model decides this entirely from the tool *descriptions*, which means description quality is behaviour quality.

Chaining is the harder case: one question that needs several tools in dependency order. That's where the most interesting finding showed up.

The three-source problem

I tried the question that motivated the whole project: **"Does my plan (`acct_123`) include priority support?"**

The agent did the sensible thing — it called `get_account_status` (which returns the plan: *Pro*) *and* `search_docs` (to find what *Pro* includes). And it *still couldn't answer*. The doc search came back with irrelevant passages, and — crucially — the agent **refused to guess** and offered to escalate.

That refusal was correct, but the answer was useless, and it taught me the real lesson:

*You can have two tools and still not answer a question, because **no source contains the join**. The account tool knew the plan name; the docs knew features; nothing connected "Pro" to "includes priority support."*

"What a plan includes" is **entitlements** data. In the real world it lives in a billing/entitlements service, *not* in the product documentation. Searching the docs for it was the wrong tool for the job, baked into my scenario. The fix was a third source — `get_plan_features(plan)` — and only then did the chain complete:

```
[step 1] get_account_status(acct_123) -> plan: Pro
[step 2] get_plan_features(Pro)         -> priority_support: true
Agent: Yes – your Pro plan includes priority support.
```

Note the **dependency order**: step 2 couldn't run until step 1 revealed the plan was *Pro*. When two lookups are independent the model fires them in parallel; when one feeds the other, it chains — and I didn't program either behaviour, it derived both from the tool descriptions. The FDE-flavoured takeaway: when an agent can't answer, ask whether *any* source even holds the

answer before blaming retrieval.

Failing gracefully has two halves

This was the part I most underestimated. In production, tools fail — accounts aren't found, arguments are wrong, a service throws, a search returns nothing. An agent that crashes or hallucinates on failure is not shippable. Robust failure needs **two separate halves**, and you need both:

Half 1 — your code never crashes. The tool dispatcher catches everything and returns an error *dict* instead of raising:

```
def run_tool(name, tool_input):
    try:
        return TOOL_FUNCTIONS[name](**tool_input)
    except TypeError as e:
        # wrong/missing args
        return {"error": "bad_arguments", "detail": str(e)}
    except Exception as e:
        # anything else
        return {"error": "tool_exception", "detail": str(e)}
```

Half 2 — the model is *told* it failed. A program that doesn't crash isn't enough; the model has to know the tool failed so it can adapt. I tag the result with `is_error: true`:

```
is_error = isinstance(result, dict) and "error" in result
tool_results.append({"type": "tool_result", "tool_use_id": tu.id,
                    "content": json.dumps(result), "is_error": is_error})
```

Why both? **Miss half 1 and the app crashes. Miss half 2 and the model treats** `{"error": "not_found"}` **as a fact and hallucinates around it** — "your account is on the Free plan" when really it wasn't found. With both in place, an unknown account produces a polite "I couldn't find that ID, please double-check or I'll escalate" — no invented data.

I also hardened the loop itself: a guard that escalates if the model repeats an identical tool call (a stuck loop), and a max-steps fallback — both surfacing a customer-facing escalation message rather than a leaked developer string.

Guardrails: do the safe part, refuse the unsafe part

An agent that can act can do harm. My rule from day one: it may **read** account state but never take destructive or billing actions (refunds, plan changes, cancellations). The interesting test is a *mixed* request:

"What plan is acct_789 on, **and please cancel its subscription.**"

The right behaviour isn't to refuse the whole thing or do the whole thing — it's to **split it**: do the allowed read (Enterprise, past due), then refuse the cancellation and escalate to a human, in the same turn. A real support agent has to answer what's safe and decline what isn't, together.

Memory — and the trap in bounding it

A real support chat is a conversation. "What plan am I on?" → "Pro" → "does *it* include SSO?" has to resolve *my* plan without re-asking. So a conversation owns the message history across

turns; that persistent list is the memory.

But memory has a cost: every turn re-sends the whole history, so tokens and money climb without limit. I bounded it to the last few turns — and hit a subtle trap. When the agent uses a tool, the history holds a **pair** joined by an id: the request (`tool_use`) and its answer (`tool_result`). The API rejects any conversation containing a `tool_result` whose `tool_use` is missing — an **orphaned result**. Naively dropping the oldest messages can cut right between them and crash the next call.

The fix: only ever trim at a **customer-turn boundary** (a real user message), never a tool-result turn — so a `tool_use/result` pair is always kept or dropped together.

Bounding agent memory isn't "keep the last N messages" — that orphans tool results. It's "keep the last N complete exchanges." Cutting on the right boundary is the whole trick.

Observability: when the data made the decision

The last piece is the one that turns "it works" into "it's operable." Every run can emit a structured **trace** — each step, each tool call (with `is_error`), token counts from the API's own usage field, latency, and cost (tokens × per-model rates). A small viewer renders it:

```
Question : Does my plan acct_123 include priority support?
step 1 [tool_use] 1006 in / 77 out $0.00696 2172ms
      → get_account_status(...) ✓ { ... "plan": "Pro" ... }
step 2 [tool_use] 1126 in / 69 out $0.00736 1556ms
      → get_plan_features(...) ✓ { ... "priority_support": true ... }
step 3 [end_turn] 1249 in / 117 out $0.00917 2614ms
SUMMARY answered · 3 steps · 3381 in / 263 out tokens · $0.023480 · 6343ms
```

Two things jump straight out of the numbers. **Input tokens climb every step** (1006 → 1126 → 1249) even though the question never changed — because the whole transcript is re-sent each loop. That's the exact cost curve my memory bound exists to cap; I could see the reason for that design decision in the trace. And **input dominates** output (3381 vs 263), so the lever for cost isn't shorter answers — it's shorter context (or caching).

I built this tracing by hand (about 50 lines) rather than reaching for a framework like Arize Phoenix or Langfuse, for the same reason I wrote the agent loop by hand: to understand the data model completely. The trade is real — a framework gives you a clickable UI and cost calc in two lines — but having built it myself, those tools are legible to me instead of magic. Build it by hand to learn; reach for the framework in production.

What I'd do next

- Replace the mock account/entitlements tools with real service calls (the agent code doesn't change — that's the point of the tool boundary).
- A hybrid keyword+vector retriever for `search_docs`.
- Wire the framework version into Phoenix for a side-by-side with my hand-rolled trace.
- Summarise trimmed-out turns into a short memory note instead of dropping them outright.

The one-paragraph version

I built a B2B support agent as a model-driven loop with three tools — account status, doc search (reusing my RAG project), and plan entitlements. It routes simple questions to one tool and chains tools when a question needs several, in dependency order. The focus was production-readiness: it never crashes on tool failure, it's *told* when a tool fails so it adapts instead of hallucinating, it splits mixed requests (safe reads, refused actions), it remembers conversations within a bounded window, and every run emits a costed, timed trace. Proven by an 8-scenario suite covering routing, chaining, and every failure mode.

Code and the full scenario suite are in the repo. The agent is built three ways (manual loop, SDK tool runner, and a framework) so the loop mechanics are visible at every level of abstraction.